

Sztuczna inteligencja - Lista 4

Mikołaj Kamiński 280596

```
In [44]: import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, MinMaxScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
import os
import glob
from pathlib import Path
from datetime import datetime
import seaborn as sns
import matplotlib.pyplot as plt
```

Ładowanie i screening danych

Zaczynamy od załadowania danych do DataFrame oraz wyciągnięcia kolumn potrzebnych w dalszej analizie.

```
In [45]: df = pd.read_csv("data/cirrhosis.csv")
valid_columns = [
    "Drug", "Age", "Sex", "Ascites", "Hepatomegaly", "Spiders",
    "Edema", "Bilirubin", "Cholesterol", "Albumin", "Copper",
    "Alk_Phos", "SGOT", "Tryglicerides", "Platelets",
    "Prothrombin", "Stage", "Status"
]
df = df[valid_columns]
print("Rozmiar dataframe", df.shape)
print("Przykładowe w dataframie")
print(df.head())
```

Rozmiar dataframe (418, 18)

Przykładowe w dataframie

	Drug	Age	Sex	Ascites	Hepatomegaly	Spiders	Edema	Bilirubin	\
0	D-penicillamine	21464	F	Y	Y	Y	Y	14.5	
1	D-penicillamine	20617	F	N	Y	Y	N	1.1	
2	D-penicillamine	25594	M	N	N	N	S	1.4	
3	D-penicillamine	19994	F	N	Y	Y	S	1.8	
4	Placebo	13918	F	N	Y	Y	N	3.4	

	Cholesterol	Albumin	Copper	Alk_Phos	SGOT	Tryglicerides	Platelets	\
0	261.0	2.60	156.0	1718.0	137.95	172.0	190.0	
1	302.0	4.14	54.0	7394.8	113.52	88.0	221.0	
2	176.0	3.48	210.0	516.0	96.10	55.0	151.0	
3	244.0	2.54	64.0	6121.8	60.63	92.0	183.0	
4	279.0	3.53	143.0	671.0	113.15	72.0	136.0	

	Prothrombin	Stage	Status
0	12.2	4.0	D
1	10.6	3.0	C
2	12.0	4.0	D
3	10.3	4.0	D
4	10.9	3.0	CL

```
In [46]: print(df.describe(include='all'))
```

	Drug	Age	Sex	Ascites	Hepatomegaly	Spiders	Edema	\
count	312	418.000000	418	312	312	312	418	
unique	2	NaN	2	2	2	2	3	
top	D-penicillamine	NaN	F	N	Y	N	N	
freq	158	NaN	374	288	160	222	354	
mean	NaN	18533.351675	NaN	NaN	NaN	NaN	NaN	
std	NaN	3815.845055	NaN	NaN	NaN	NaN	NaN	
min	NaN	9598.000000	NaN	NaN	NaN	NaN	NaN	
25%	NaN	15644.500000	NaN	NaN	NaN	NaN	NaN	
50%	NaN	18628.000000	NaN	NaN	NaN	NaN	NaN	
75%	NaN	21272.500000	NaN	NaN	NaN	NaN	NaN	
max	NaN	28650.000000	NaN	NaN	NaN	NaN	NaN	

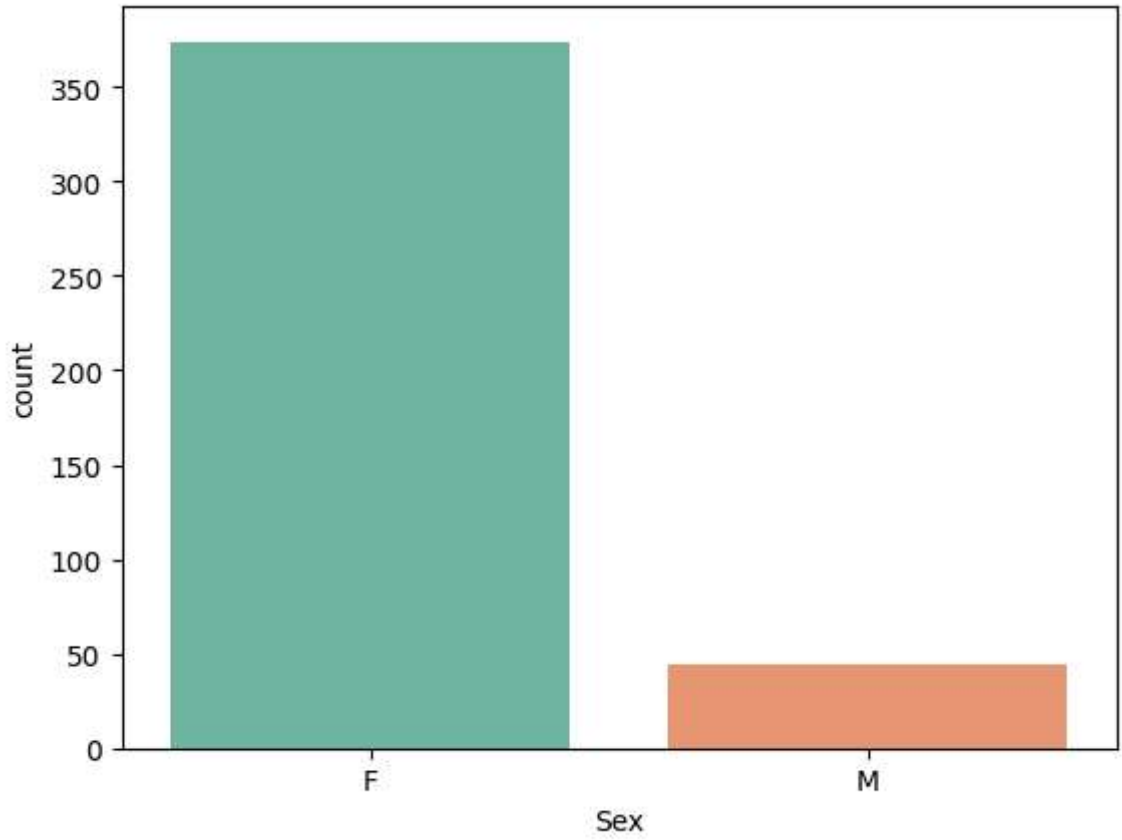
	Bilirubin	Cholesterol	Albumin	Copper	Alk_Phos	\
count	418.000000	284.000000	418.000000	310.000000	312.000000	
unique	NaN	NaN	NaN	NaN	NaN	
top	NaN	NaN	NaN	NaN	NaN	
freq	NaN	NaN	NaN	NaN	NaN	
mean	3.220813	369.510563	3.497440	97.648387	1982.655769	
std	4.407506	231.944545	0.424972	85.613920	2140.388824	
min	0.300000	120.000000	1.960000	4.000000	289.000000	
25%	0.800000	249.500000	3.242500	41.250000	871.500000	
50%	1.400000	309.500000	3.530000	73.000000	1259.000000	
75%	3.400000	400.000000	3.770000	123.000000	1980.000000	
max	28.000000	1775.000000	4.640000	588.000000	13862.400000	

	SGOT	Tryglicerides	Platelets	Prothrombin	Stage	Status
count	312.000000	282.000000	407.000000	416.000000	412.000000	418
unique	NaN	NaN	NaN	NaN	NaN	3
top	NaN	NaN	NaN	NaN	NaN	C
freq	NaN	NaN	NaN	NaN	NaN	232
mean	122.556346	124.702128	257.024570	10.731731	3.024272	NaN
std	56.699525	65.148639	98.325585	1.022000	0.882042	NaN
min	26.350000	33.000000	62.000000	9.000000	1.000000	NaN
25%	80.600000	84.250000	188.500000	10.000000	2.000000	NaN
50%	114.700000	108.000000	251.000000	10.600000	3.000000	NaN
75%	151.900000	151.000000	318.000000	11.100000	4.000000	NaN
max	457.250000	598.000000	721.000000	18.000000	4.000000	NaN

Analizując dane możemy zauważyć kilka ciekawych własności.

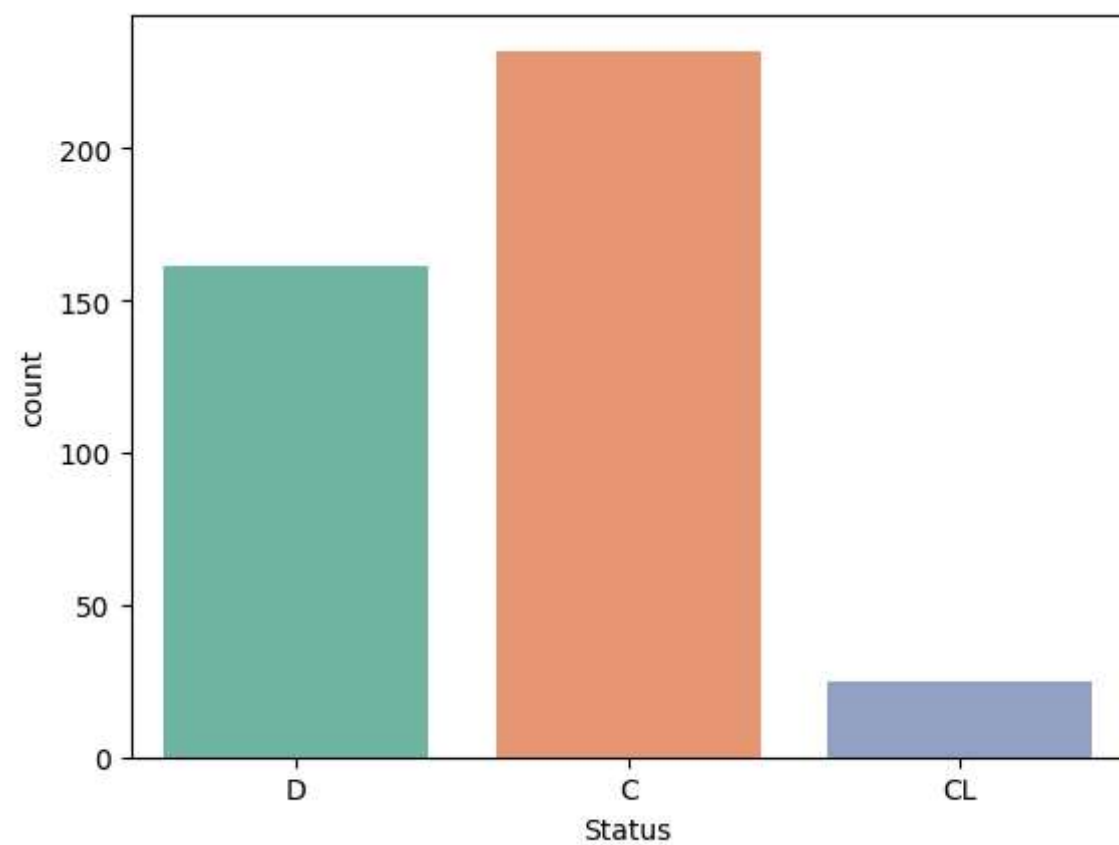
Prawie całość pacjentów w datasetcie stanowią kobiety, co może obniżyć skuteczność modeli jeżeli chodzi o wyniki dla mężczyzn

```
In [47]: ax = sns.countplot(data = df, x='Sex', hue="Sex", palette="Set2")
```



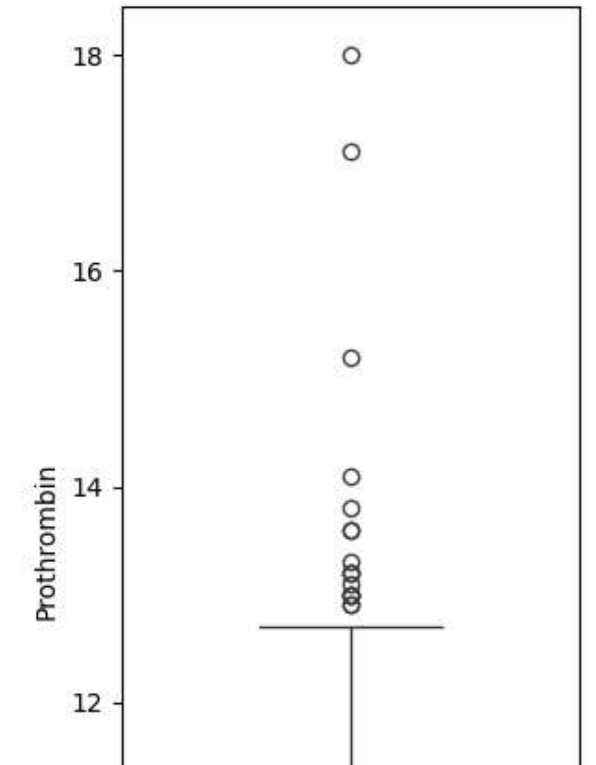
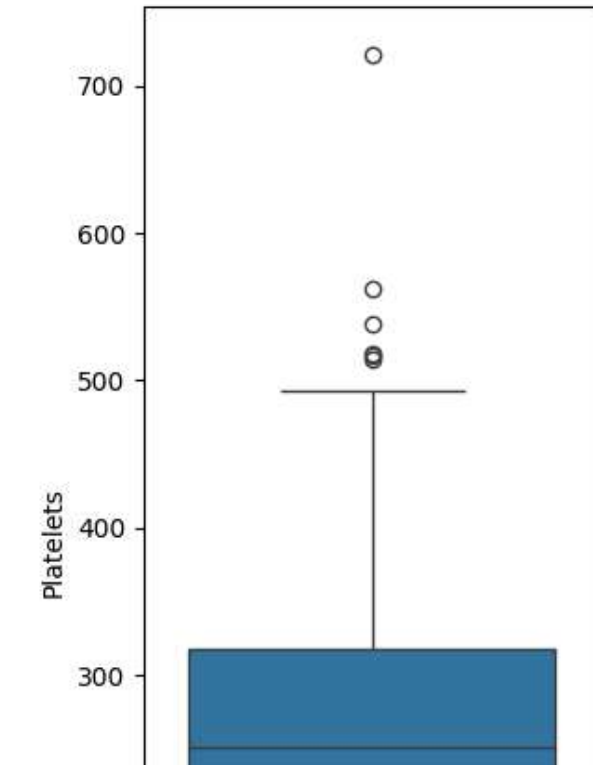
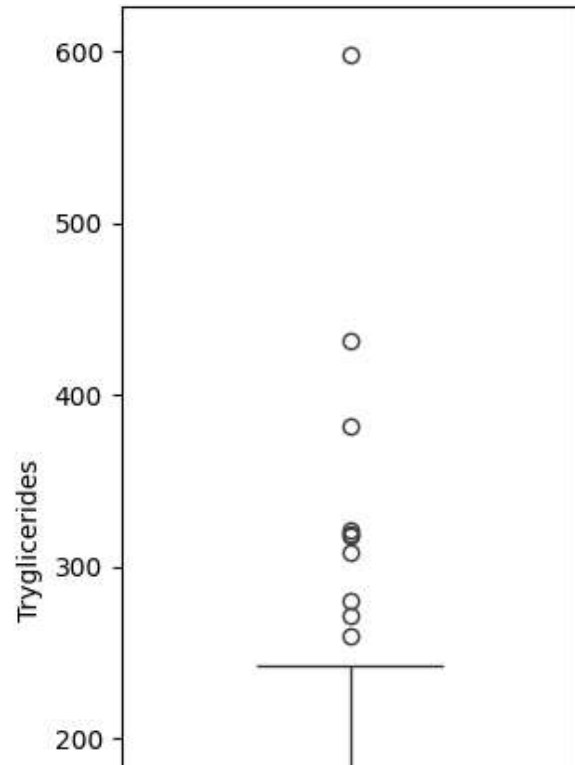
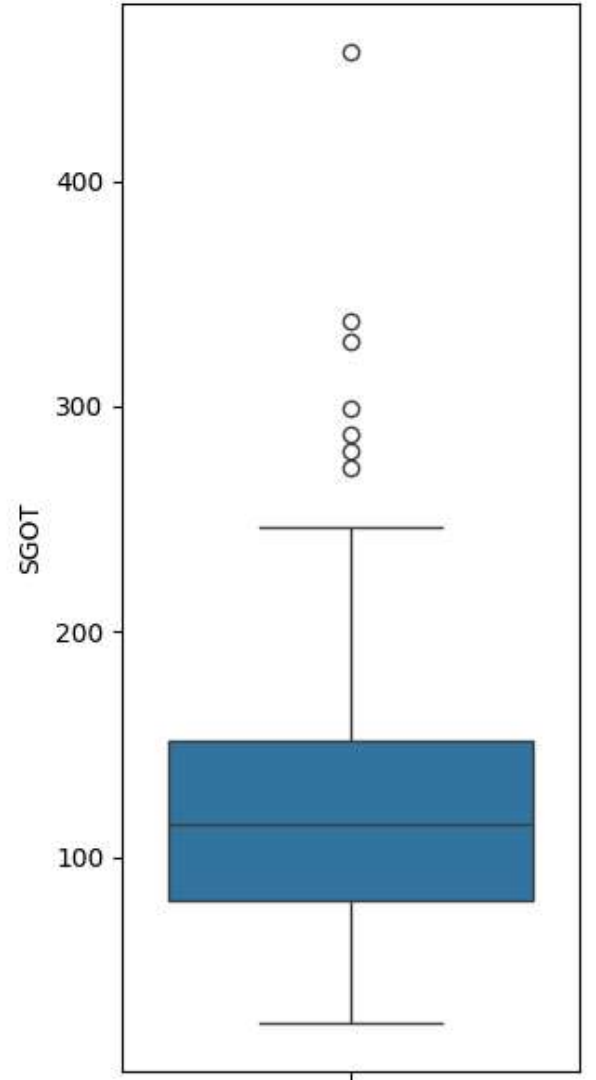
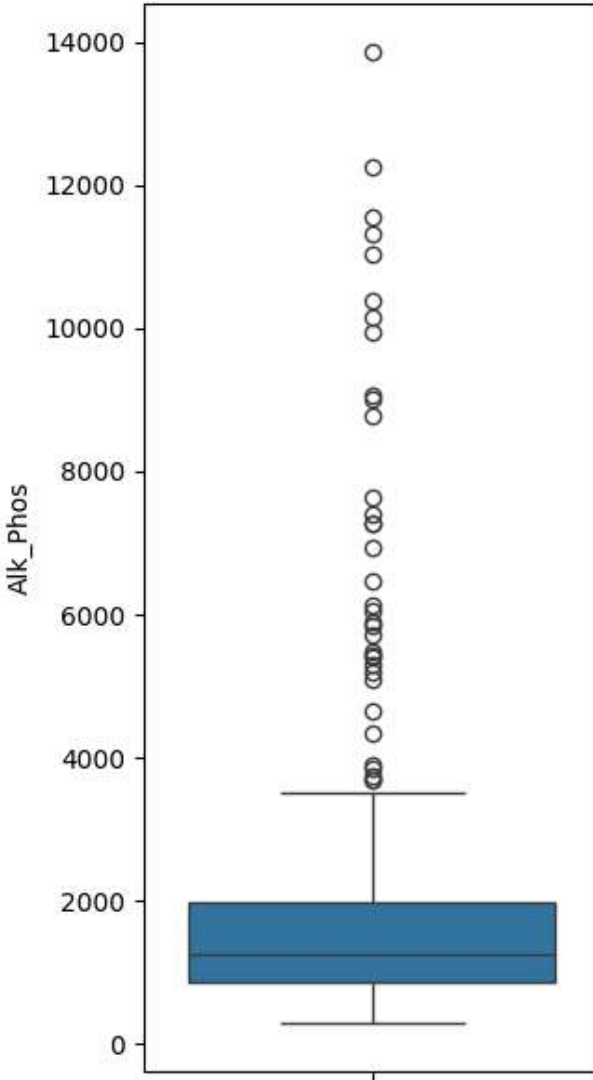
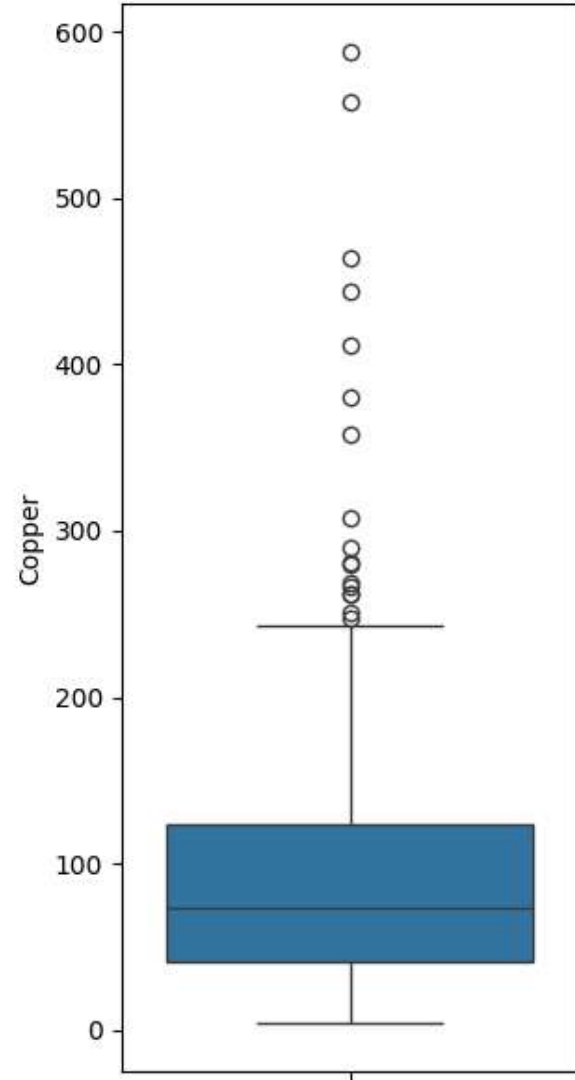
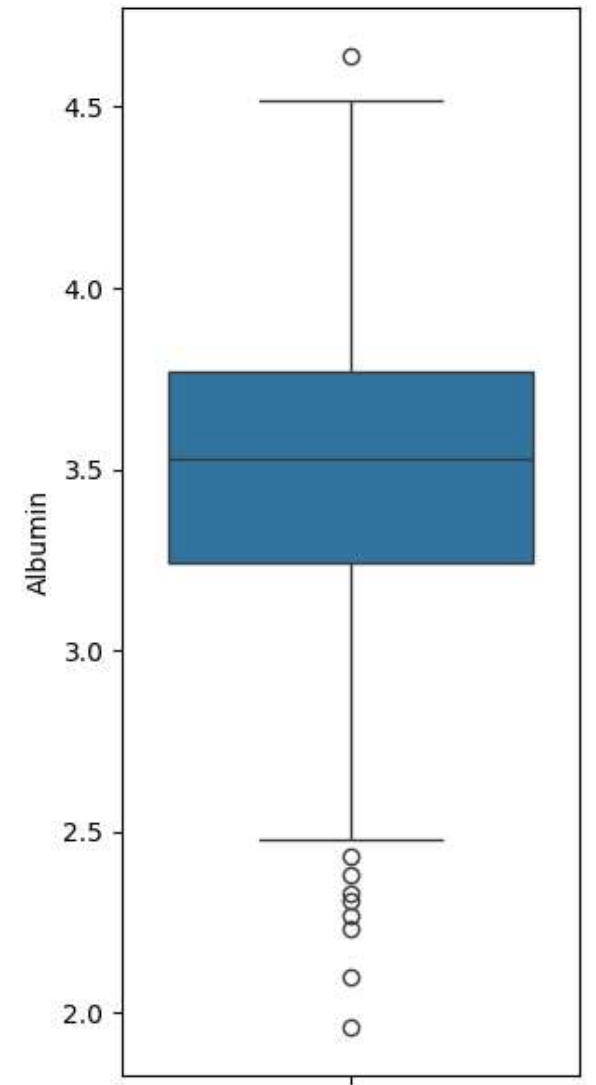
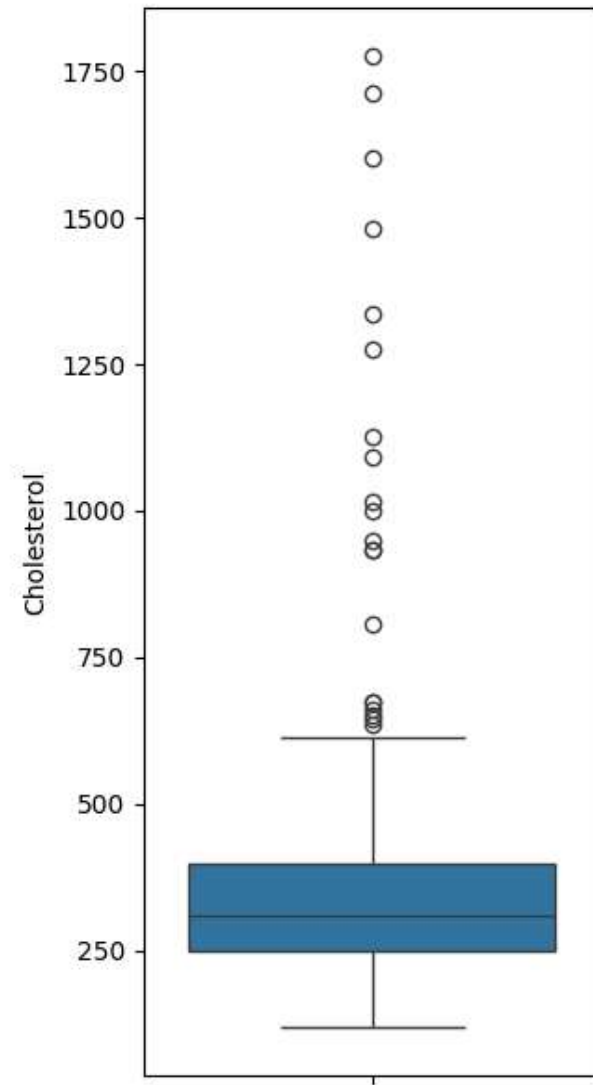
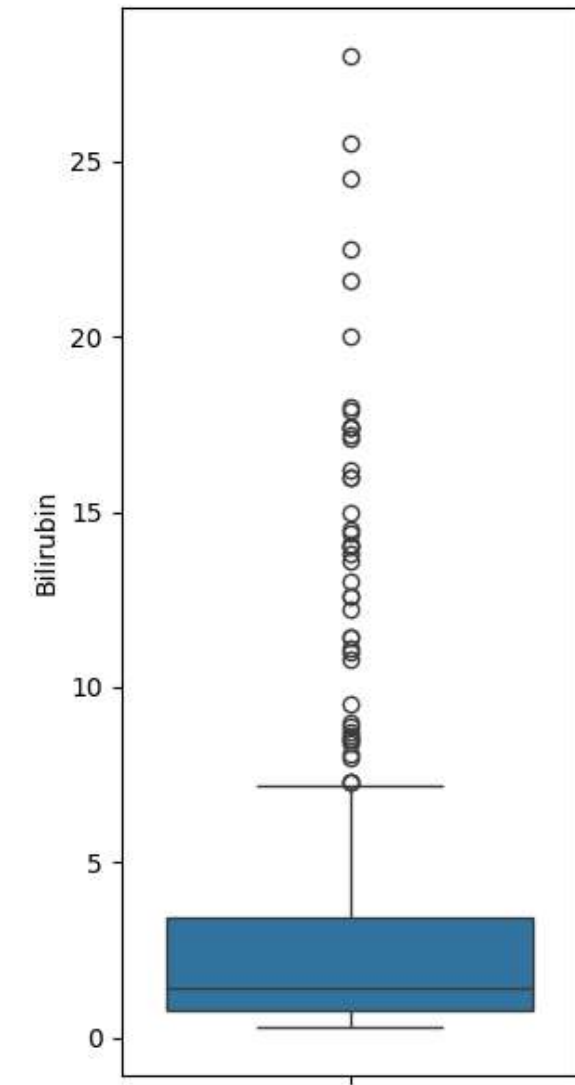
Zmienna docelowa jest wyraźnie niezbalansowana, isntnieje wyraźna przewaga klasy C nad stanami krytycznymi (D - śmierć, CL - przeszczep).

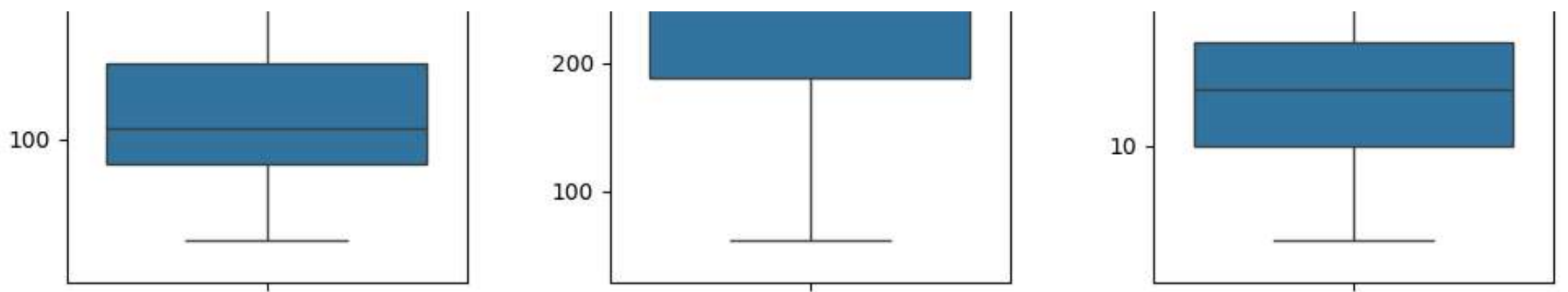
```
In [48]: ax = sns.countplot(data=df, x="Status",hue="Status", palette="Set2")
```



Większość miar związanych z wynikami krwi, posiada wielu otlierów. W skrajnych przypadkach wykorzystywane średniej zamiast mediany do uzupełniania danych może spowodować spadek skuteczności modelu.

```
In [49]: metric_columns = [  
    "Bilirubin", "Cholesterol", "Albumin", "Copper",  
    "Alk_Phos", "SGOT", "Tryglicerides", "Platelets",  
    "Prothrombin"  
]  
fig, axs = plt.subplots(3,3, figsize=(10, 18))  
for i in range(9):  
    rw = i // 3  
    cl = i % 3  
    sns.boxplot(data=df, y=metric_columns[i], ax=axs[rw][cl])  
plt.tight_layout()  
plt.show()
```

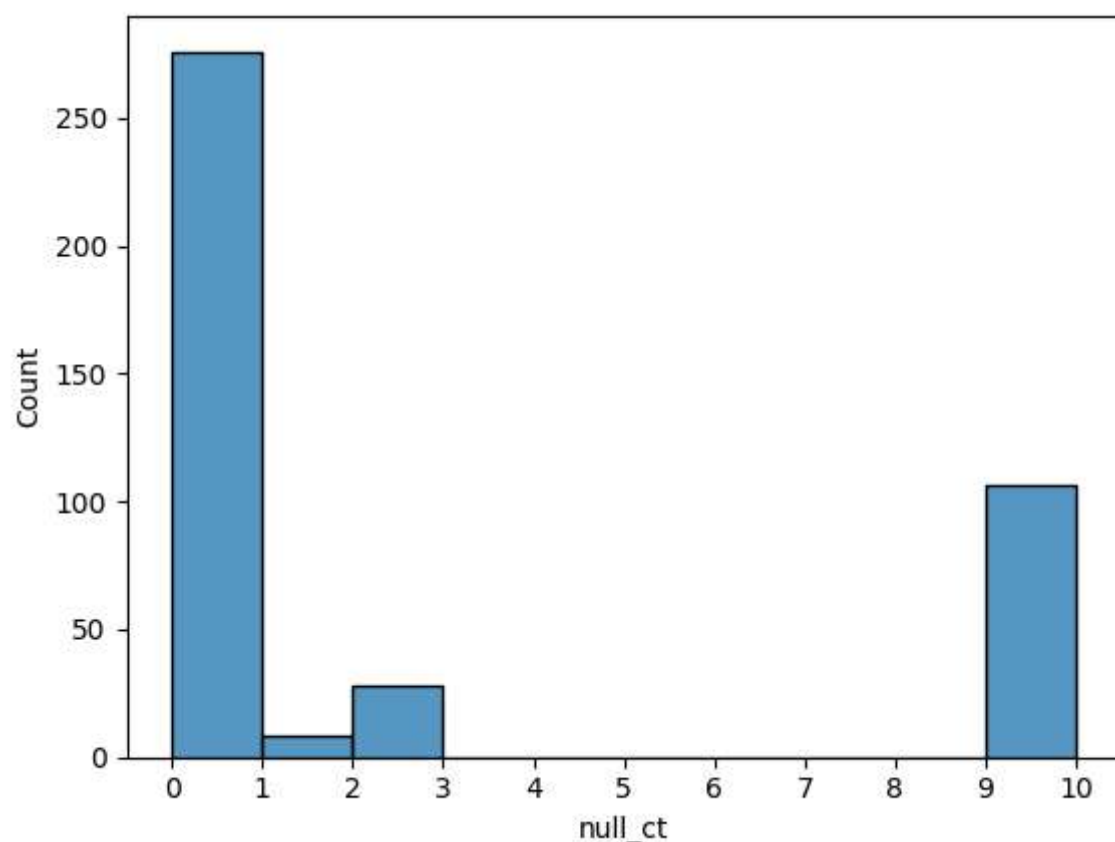




Zbiór zawiera ponad 100 rekordów gdzie wybrakowane kolumny to aż 9/17. O ile dane z kilkoma wartościami wnoszą dużą wartość to dane które są tworzone zupełnie syntetycznie, mogą utrudniać sensowne trenowanie modelu i tworzyć szum. W ramach eksperymentu postanowiłem utworzyć trzy zbiory: bazowy(brak zmian), średnio wyczyszczony(usunięte tylko wiersze z 9 lub więcej nullami). agresywnie wyczyszczony(usunięto wszystkie wiersze z wartościami null)

```
In [50]: from matplotlib.ticker import MaxNLocator
```

```
df['null_ct'] = (df.isna().apply(lambda x: x.sum(), axis=1))
ax = sns.histplot(data=df['null_ct'], bins=10)
plt.xticks(range(int(df['null_ct'].min()), int(df['null_ct'].max()) + 1))
plt.show()
df = df.drop(columns='null_ct')
```



```
In [51]: df_cleaned = df.dropna(axis=0, thresh=10, inplace=False)
df_cleaned_more = df.dropna(axis=0, how='any', inplace=False)
dataframes = [df, df_cleaned, df_cleaned_more]
for dataF in dataframes:
    print(dataF.shape)
```

```
(418, 18)
(312, 18)
(276, 18)
```

Kolejnym etapem jest budowa Pipeline, który pozwoli na zautomatyzowaną transformację danych. Do optymalizacji hiperparametrów wykorzystano algorytm GridSearchCV, dzięki czemu wyłoniono zestaw parametrów maksymalizujący średni wynik dla metryki `f1_macro`. Metryka ta została wybrana ze względu na to, że jest znacznie bardziej miarodajna niż `accuracy` w przypadku niezbalansowanych zbiorów danych - model zgadujący ciągle C osiągnął by `accuracy` równe 55%. W przeciwieństwie do wariantów `f1_weighted` i `f1_micro`, uśrednianie makro traktuje każdą klasę równorzędnie, co zapobiega faworyzowaniu najliczniejszej grupy pacjentów

```
In [52]: def pipeline(df: pd.DataFrame, classifier, param_grid: dict) -> pd.DataFrame:
    X = df.drop(columns='Status')
    y = df['Status']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    men_only = X_test['Sex']=='M'
    X_test_men = X_test[men_only]
    y_test_men = y_test[men_only]

    num_features = ['Age', 'Bilirubin', 'Albumin', 'Copper', 'Alk_Phos', 'SGOT', 'Tryglicerides', 'Platelets', 'Prothrombin',
cat_features = ['Drug', 'Sex', 'Ascites', 'Hepatomegaly', 'Spiders', 'Edema']

    num_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='mean')),
        ('scales', StandardScaler())
    ])

    cat_transformer = Pipeline(steps=[
```

```

        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))
    ])

    preprocessor = ColumnTransformer(
    transformers=[
        ('num', num_transformer, num_features),
        ('cat', cat_transformer, cat_features)
    ])

    model_pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', classifier)
    ])

    grid_search = GridSearchCV(
        estimator=model_pipeline,
        param_grid=param_grid,
        cv=5,
        scoring='f1_macro',
        n_jobs=-1,
        verbose=3
    )

    grid_search.fit(X_train, y_train)

    best_model = grid_search.best_estimator_
    predictions = best_model.predict(X_test)
    predictions_men = best_model.predict(X_test_men)

    acc = accuracy_score(y_test, predictions)
    prec = precision_score(y_test, predictions, average='weighted', zero_division=0)
    rec = recall_score(y_test, predictions, average='weighted', zero_division=0)
    f1 = f1_score(y_test, predictions, average='weighted', zero_division=0)

    model_name = classifier.__class__.__name__
    parameters = grid_search.best_params_

    best_num_strat = parameters.get('preprocessor__num__imputer__strategy', 'Nie testowano w Grid')
    best_cat_strat = parameters.get('preprocessor__cat__imputer__strategy', 'Nie testowano w Grid')

    results = pd.DataFrame({
        'Classifier': [model_name],
        'Brak_Num_Best': [best_num_strat],
        'Brak_Cat_Best': [best_cat_strat],
        'Accuracy': [round(acc, 4)],
        'Precision': [round(prec, 4)],
        'Recall': [round(rec, 4)],
        'F1_Score': [round(f1, 4)],
        'Hiperparametry': [str(parameters)]
    })

    report1 = classification_report(y_test, predictions, zero_division=0)
    report2 = classification_report(y_test_men, predictions_men, zero_division=0)

    return results, report1, report2

```

W swoich testach wykorzystałem 4 rodzaje klasyfikatorów:

Gaussian Naive Bayes

Algorytm probabilistyczny oparty na twierdzeniu Bayesa. Nazywany jest „naiwnym”, ponieważ opiera się na silnym założeniu o wzajemnej niezależności wszystkich cech (zmiennych objaśniających) względem zmiennej docelowej. W tym badaniu wykorzystano wariant Gaussa (GaussianNB), który zakłada, że zmienne numeryczne charakteryzują się rozkładem normalnym. Mimo swojej prostoty i "naiwnego" założenia, model ten jest niezwykle szybki, dobrze radzi sobie z brakami w danych i często stanowi doskonały punkt odniesienia (baseline) dla bardziej złożonych algorytmów.

Decision Tree Classifier

Jest to model o strukturze grafu skierowanego (przypominającego drzewo lub schemat blokowy), w którym węzły wewnętrzne reprezentują testy na poszczególnych cechach, gałęzie to wyniki tych testów, a liście to ostateczne etykiety klas (w tym przypadku status pacjenta). Algorytm buduje drzewo, iteracyjnie dzieląc zbiór danych w taki sposób, aby maksymalizować zysk informacyjny lub minimalizować zanieczyszczenie. Główną zaletą drzew decyzyjnych jest ich wysoka interpretowalność, jednak mają one silną tendencję do przeuczenia, przez co konieczne jest stosowanie pruningu, np. poprzez ograniczanie maksymalnej głębokości drzewa.

Random Forest Classifier

Las losowy to zaawansowany algorytm zespołowy, który stanowi rozwinięcie koncepcji drzew decyzyjnych i skutecznie rozwiązuje ich problem przeuczenia. Model ten buduje wiele niezależnych drzew decyzyjnych na podstawie losowych podzbiorów danych treningowych oraz przy użyciu losowych podzbiorów cech podczas każdego podziału węzła. Ostateczna predykcja lasu losowego jest wynikiem głosowania większościowego wszystkich wygenerowanych drzew. Dzięki temu podejściu model charakteryzuje się znacznie wyższą dokładnością, stabilnością oraz odpornością na szum w danych.

Support Vector Machine - SVM

Jeden z najpotężniejszych algorytmów klasyfikacyjnych. Jego głównym celem jest znalezienie optymalnej hiperpłaszczyzny w wielowymiarowej przestrzeni cech, która oddziela od siebie poszczególne klasy z maksymalnym możliwym marginesem (odległością od najbliższych punktów danych, zwanych wektorami nośnymi). W podstawowej wersji SVM służy do klasyfikacji liniowej, jednak dzięki zastosowaniu tzw.kernel trick – polegającej na transformacji danych do przestrzeni o wyższym wymiarze – potrafi niezwykle skutecznie rozwiązywać problemy, w których dane nie są liniowo separowalne.

```
In [53]: save_path = Path(os.getcwd()) / "test_result"
now = "20260525_002030"
```

```
In [ ]: tree_param = {
    'preprocessor__num__imputer__strategy': ['mean', 'median'],
    'preprocessor__cat__imputer__strategy': ['most_frequent', 'constant'],
    'classifier__max_depth': [3, 5, 10, None],
    'classifier__min_samples_split': [2, 10],
    'preprocessor__num__scales': ['passthrough', StandardScaler(), MinMaxScaler()]
}

bayes_param = {
    'preprocessor__num__imputer__strategy': ['mean', 'median'],
    'preprocessor__cat__imputer__strategy': ['most_frequent', 'constant'],
    'classifier__var_smoothing': [1e-9, 1e-8, 1e-7],
    'preprocessor__num__scales': ['passthrough', StandardScaler(), MinMaxScaler()]
}

forest_param = {
    'preprocessor__num__imputer__strategy': ['mean', 'median'],
    'preprocessor__cat__imputer__strategy': ['most_frequent', 'constant'],
    'classifier__n_estimators': [50, 100, 200],
    'classifier__max_depth': [None, 5, 10],
    'classifier__min_samples_split': [2, 10],
    'preprocessor__num__scales': ['passthrough', StandardScaler(), MinMaxScaler()],
}

svm_param = {
    'preprocessor__num__imputer__strategy': ['mean', 'median'],
    'preprocessor__cat__imputer__strategy': ['most_frequent', 'constant'],
    'classifier__C': [0.1, 1, 10],
    'classifier__kernel': ['linear', 'rbf'],
    'classifier__gamma': ['scale', 'auto'],
    'preprocessor__num__scales': ['passthrough', StandardScaler(), MinMaxScaler()]
}

model_params = [
    (DecisionTreeClassifier(random_state=42), tree_param),
    (GaussianNB(), bayes_param),
    (RandomForestClassifier(random_state=42), forest_param),
    (SVC(), svm_param)
]

test_res = pd.DataFrame()

# Prepare separate report files per dataframe kind
reports_dir = Path(os.getcwd()) / "test_result" / "reports"
os.makedirs(reports_dir, exist_ok=True)
# now = datetime.now().strftime('%Y%m%d_%H%M%S')
report_files = {
    'base': reports_dir / f"classification_reports_base_{now}.txt",
    'medium_cleaned': reports_dir / f"classification_reports_medium_cleaned_{now}.txt",
    'hard_cleaned': reports_dir / f"classification_reports_hard_cleaned_{now}.txt",
}

# create files with headers
for k, p in report_files.items():
    with open(p, 'w', encoding='utf-8') as f:
        f.write(f"Classification reports for {k} generated on {datetime.now().isoformat()}\n\n")

for clss, params in model_params:
    for num, dataframe in enumerate(dataframes, 0):
        res, report_overall, report_men = pipeline(df=dataframe, classifier=clss, param_grid=params)
        if num == 0:
            df_kind = 'base'
        elif num == 1:
            df_kind = 'medium_cleaned'
        elif num == 2:
            df_kind = 'hard_cleaned'
```

```

else:
    df_kind = ''
res['df_kind'] = df_kind
test_res = pd.concat([test_res, res], ignore_index=True)

model_name = res['Classifier'].iloc[0] if 'Classifier' in res.columns else cls.__class__.__name__
hiperparams = res['Hiperparametry'].iloc[0] if 'Hiperparametry' in res.columns else ''

# append this model's reports to the corresponding file based on dataframe kind
target_file = report_files.get(df_kind, reports_dir / report_files[df_kind])
with open(target_file, 'a', encoding='utf-8') as f:
    f.write('=== ' + model_name + ' - ' + df_kind + ' ===\n')
    f.write('Hiperparametry: ' + str(hiperparams) + '\n\n')
    f.write('--- WYNIKI DLA CAŁEGO ZBIORU TESTOWEGO ---\n')
    f.write(report_overall + '\n')
    f.write('--- WYNIKI TYLKO DLA MĘŻCZYZN ---\n')
    f.write(report_men + '\n\n')

# Save res
os.makedirs(save_path, exist_ok=True)
test_res.to_csv(save_path / f"{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv", index=False)

```

Analiza uzyskanych wyników

```

In [54]: save_path = Path(os.getcwd()) / "test_result"
proper_file = max(glob.glob(str(save_path)+"/*.csv"), key=os.path.getctime)
analysis_df = pd.read_csv(proper_file)

```

1. Metody uzupełniania danych

```

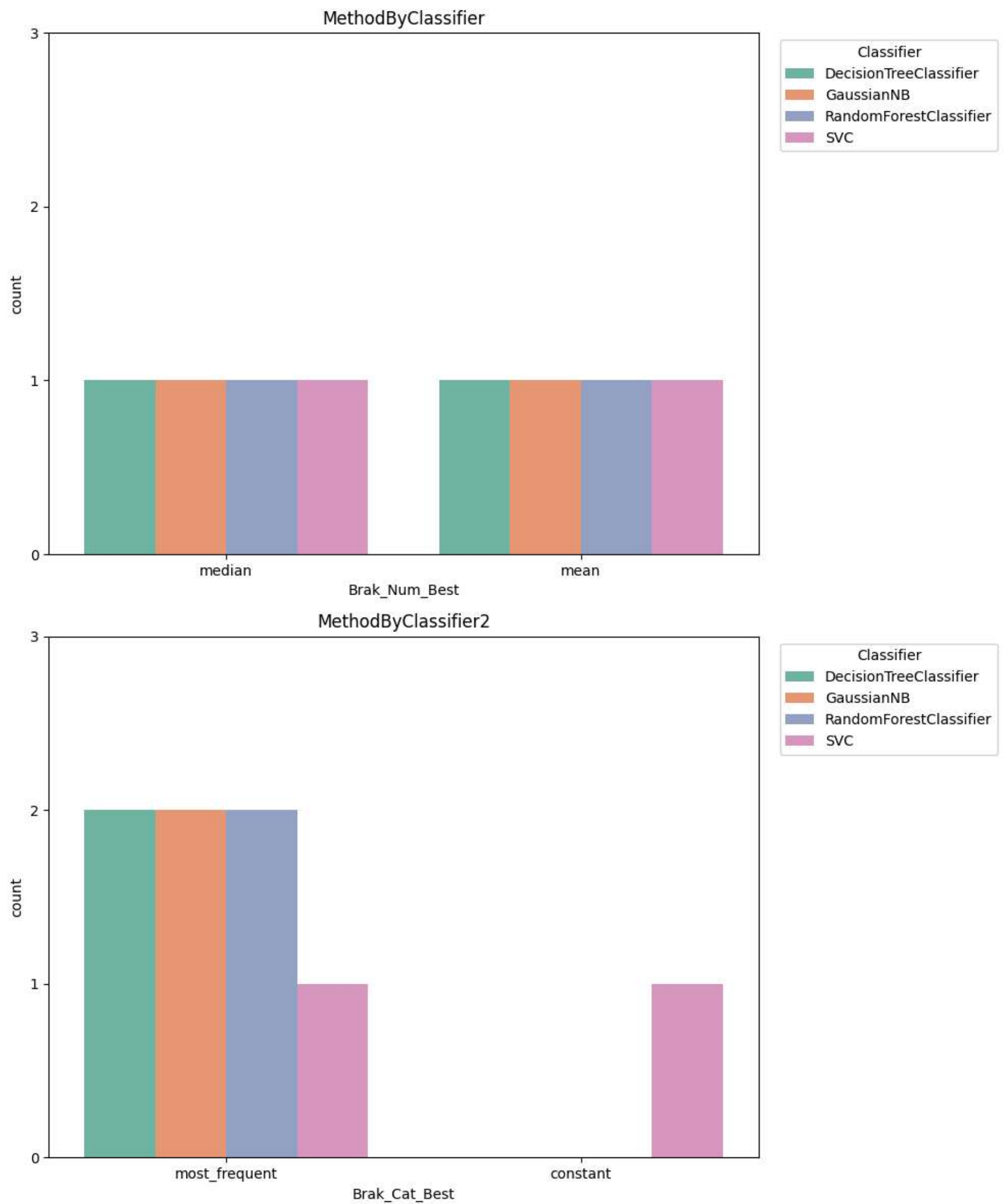
In [55]: def get_completion_method_by_dataframe(dataframe: pd.DataFrame, method_col: str, hue: str, title: str, ax=None):
    if ax is None:
        fig, ax = plt.subplots(figsize=(10, 6))
    sns.countplot(
        data=dataframe,
        x=method_col,
        hue=hue,
        palette="Set2",
        ax=ax
    )
    ax.set_title(title)
    ax.set_yticks([i for i in range(4)])
    ax.legend(title=hue.capitalize(), bbox_to_anchor=(1.02, 1), loc="upper left")
    return ax

fig, axes = plt.subplots(2, 1, figsize=(10, 12), sharey=True)
filtered = analysis_df.where(analysis_df['df_kind']!='hard_cleaned')

get_completion_method_by_dataframe(filtered, 'Brak_Num_Best', 'Classifier', 'MethodByClassifier', ax=axes[0])
get_completion_method_by_dataframe(filtered, 'Brak_Cat_Best', 'Classifier', 'MethodByClassifier2', ax=axes[1])

plt.tight_layout()
plt.show()

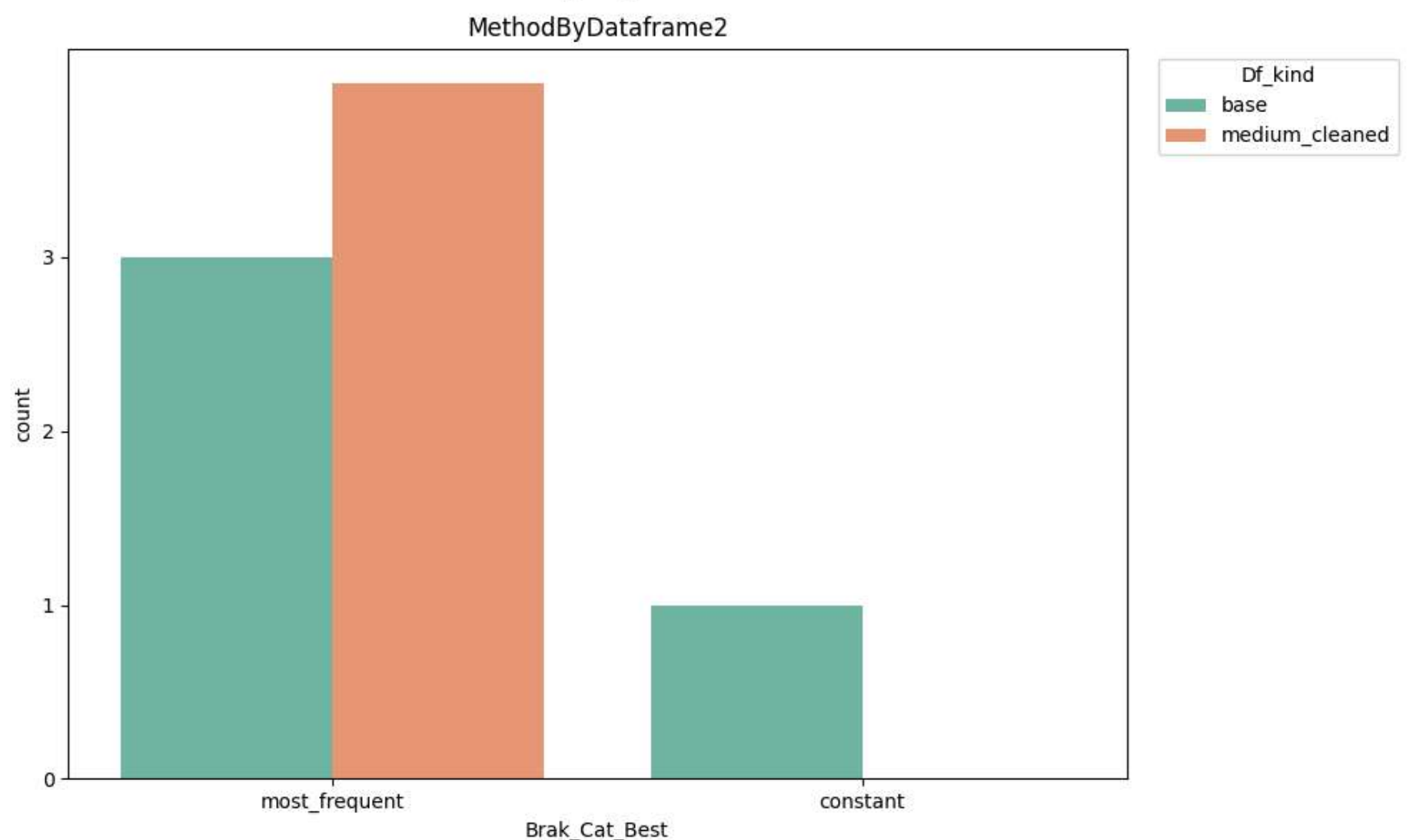
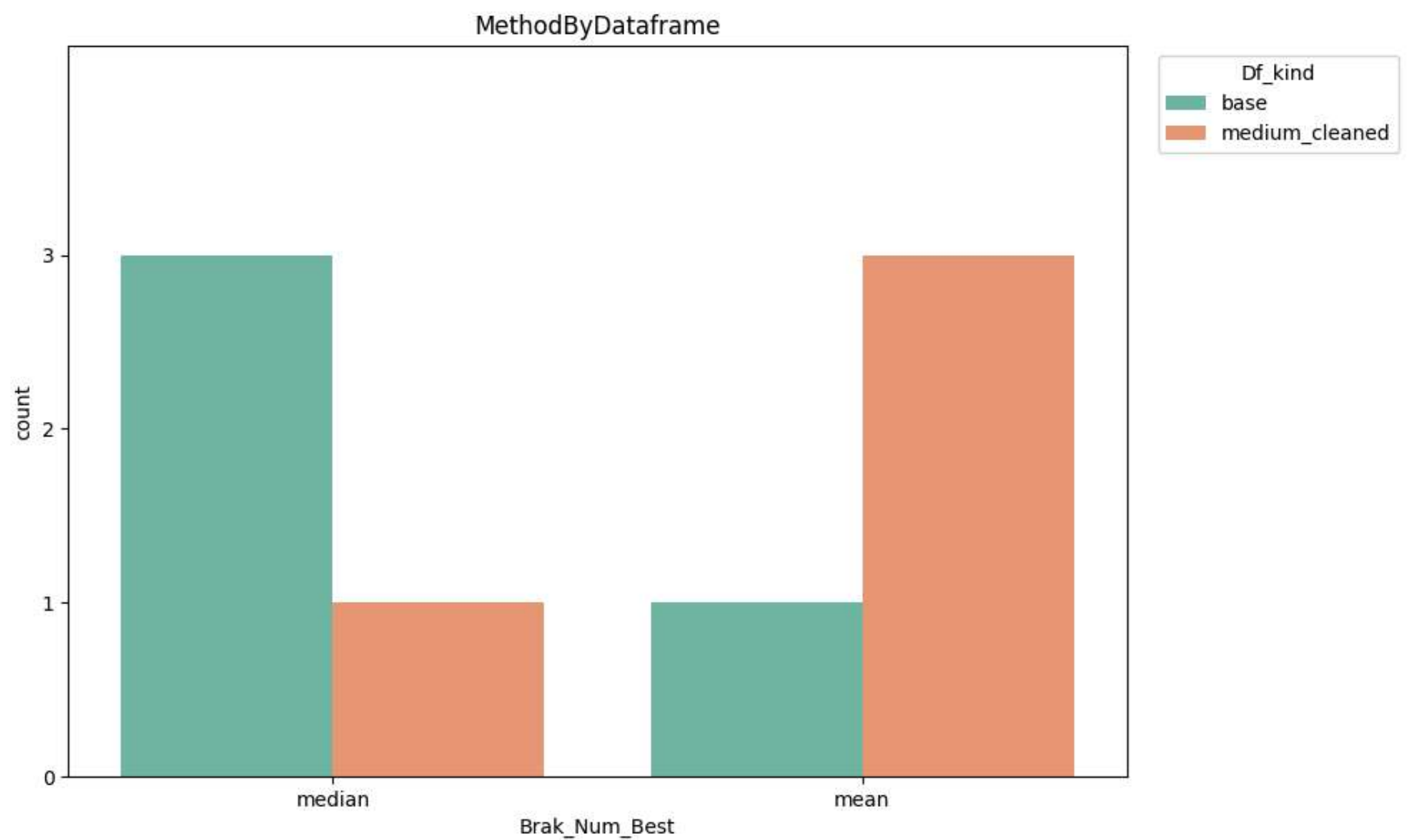
```



```
In [56]: fig, axes = plt.subplots(2, 1, figsize=(10, 12), sharey=True)

get_completion_method_by_dataframe(filtered, 'Brak_Num_Best', 'df_kind', 'MethodByDataframe', ax=axes[0])
get_completion_method_by_dataframe(filtered, 'Brak_Cat_Best', 'df_kind', 'MethodByDataframe2', ax=axes[1])

plt.tight_layout()
plt.show()
```



Ze zbioru zostały usunięte dane dotyczące uzupełniania danych w zbiorze kompletnym jako, że w zbiorze kompletnym metody uzupełniania danych nie mają żadnego znaczenia. Analizując wykresy możemy zauważyć, że większość modeli o najwyższym **f1** najczęściej występującą wartość do uzupełniania brakujących danych. O ile w przypadku podziału według klasyfikatora ciężko jest wyjąć konkretne wnioski o tyle w przypadku podziału według rodzaju użytego zbioru widzimy już znacznie ciekawszą zależność. Przy pełnym zbiorze najlepsze modele, korzystały z mediany do uzupełniania danych jednak dla średnio wyczyszczonego zbioru modele decydowały się prawie wyłącznie na średnią. Wstępna eksploracja danych oraz analiza statystyk sugerowała, że z uwagi na obecność silnych outlierów w wynikach badań krwi, najbezpieczniejszym i najskuteczniejszym wyborem powinna być mediana.

```
In [57]: metric_columns = [
    "Bilirubin", "Cholesterol", "Albumin", "Copper", "Alk_Phos",
    "SGOT", "Tryglicerides", "Platelets", "Prothrombin"
]
col = metric_columns[4]

df_mean_filled = df.copy()
df_median_filled = df.copy()
df_mean_filled_med = df_cleaned.copy()
df_median_filled_med = df_cleaned.copy()
```

```

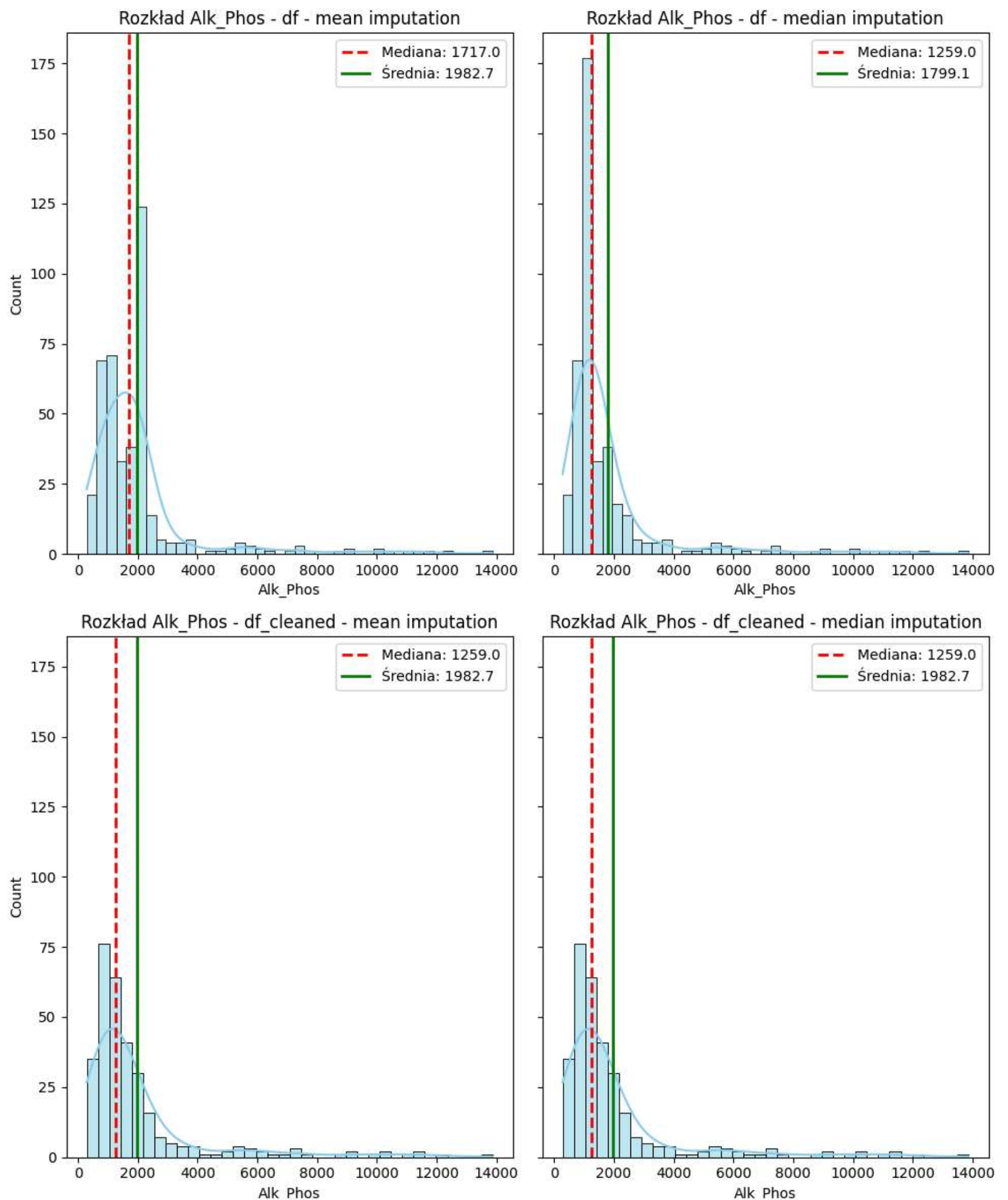
df_mean_filled[col] = df_mean_filled[col].fillna(df_mean_filled[col].mean())
df_median_filled[col] = df_median_filled[col].fillna(df_median_filled[col].median())
df_mean_filled_med[col] = df_mean_filled_med[col].fillna(df_mean_filled_med[col].mean())
df_median_filled_med[col] = df_median_filled_med[col].fillna(df_median_filled_med[col].median())

fig, axs = plt.subplots(2,2, figsize=(10, 12), sharey=True)
plots = [
    (axs[0, 0], df_mean_filled, "df - mean imputation"),
    (axs[0, 1], df_median_filled, "df - median imputation"),
    (axs[1, 0], df_mean_filled_med, "df_cleaned - mean imputation"),
    (axs[1, 1], df_median_filled_med, "df_cleaned - median imputation"),
]

for ax, data, title in plots:
    sns.histplot(data[col].dropna(), kde=True, color="skyblue", ax=ax)
    ax.axvline(data[col].median(), color="red", linestyle="dashed", linewidth=2, label=f"Mediana: {data[col].median():.1f}")
    ax.axvline(data[col].mean(), color="green", linewidth=2, label=f"Średnia: {data[col].mean():.1f}")
    ax.set_title(f"Rozkład {col} - {title}")
    ax.legend()

plt.tight_layout()
plt.show()

```



Jednak analizując wykresy dla metryki `Alk_Phos`, która według boxplotów zawierała najwięcej outlierów możemy zauważyć, że wybór uzupełnienia dla średnio oczyszczonego Dataframe'u nie ma żadnego znaczenia - wartość mediany jest identyczna dla obu rodzajów uzupełnień (podobnie jak średnia). W przypadku danych bazowych uzupełnianie średnią znacznie zawyża średnia oraz medianę co przekłada się na zaburzenie rzeczywistych danych i gorsze wyniki trenowanych modeli.

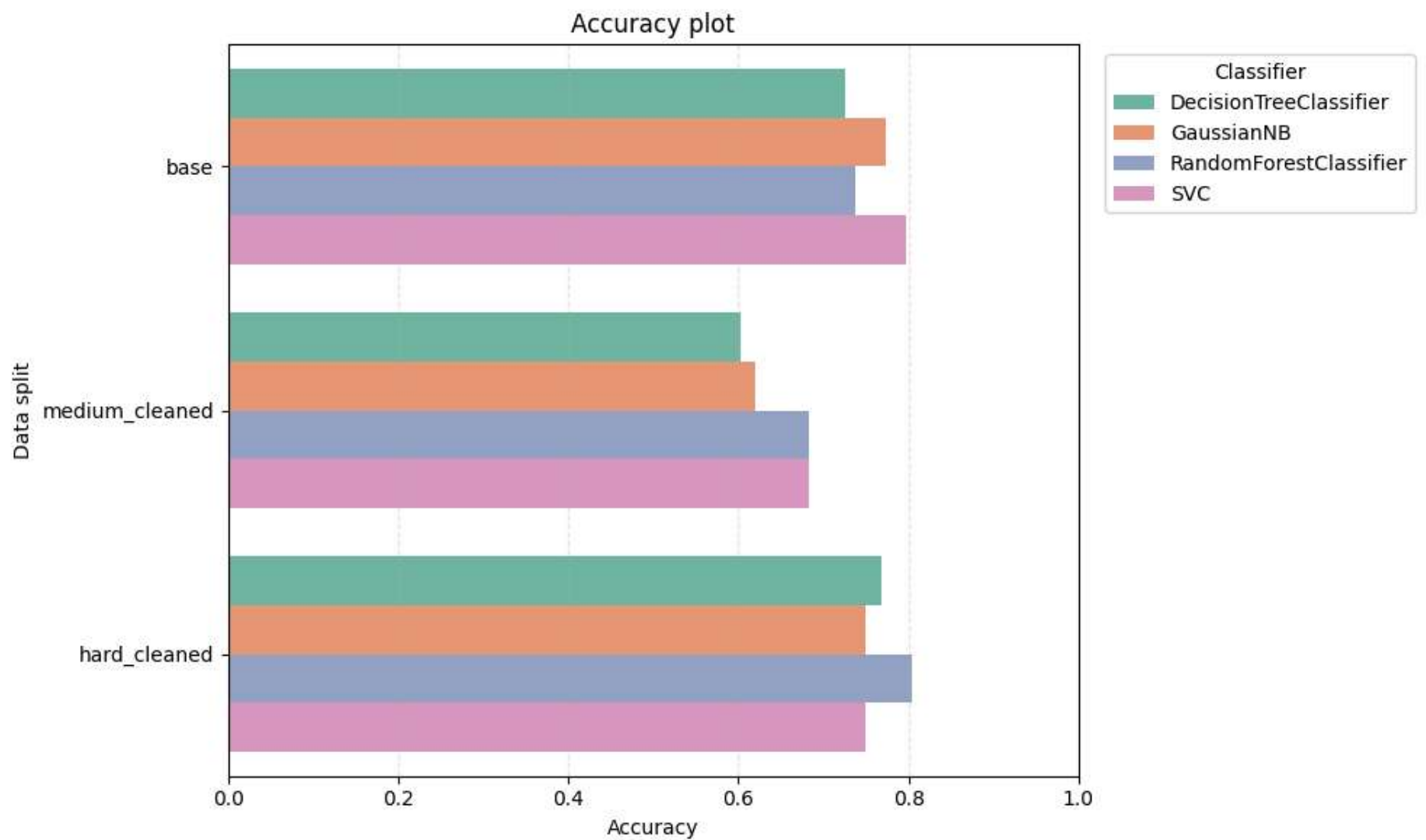
2. Średnie wartości metryk wyniki względem rodzajów klasyfikatorów i zestawu danych

```
In [58]: def metric_byDataframe_byClassifier(dataframe: pd.DataFrame, metric_name:str, title: str,palette: str):
plt.figure(figsize=(10, 6))
ax = sns.barplot(
    data=dataframe,
    x=metric_name,
    y="df_kind",
    hue="Classifier",
    orient="h",
    palette=palette
)
```

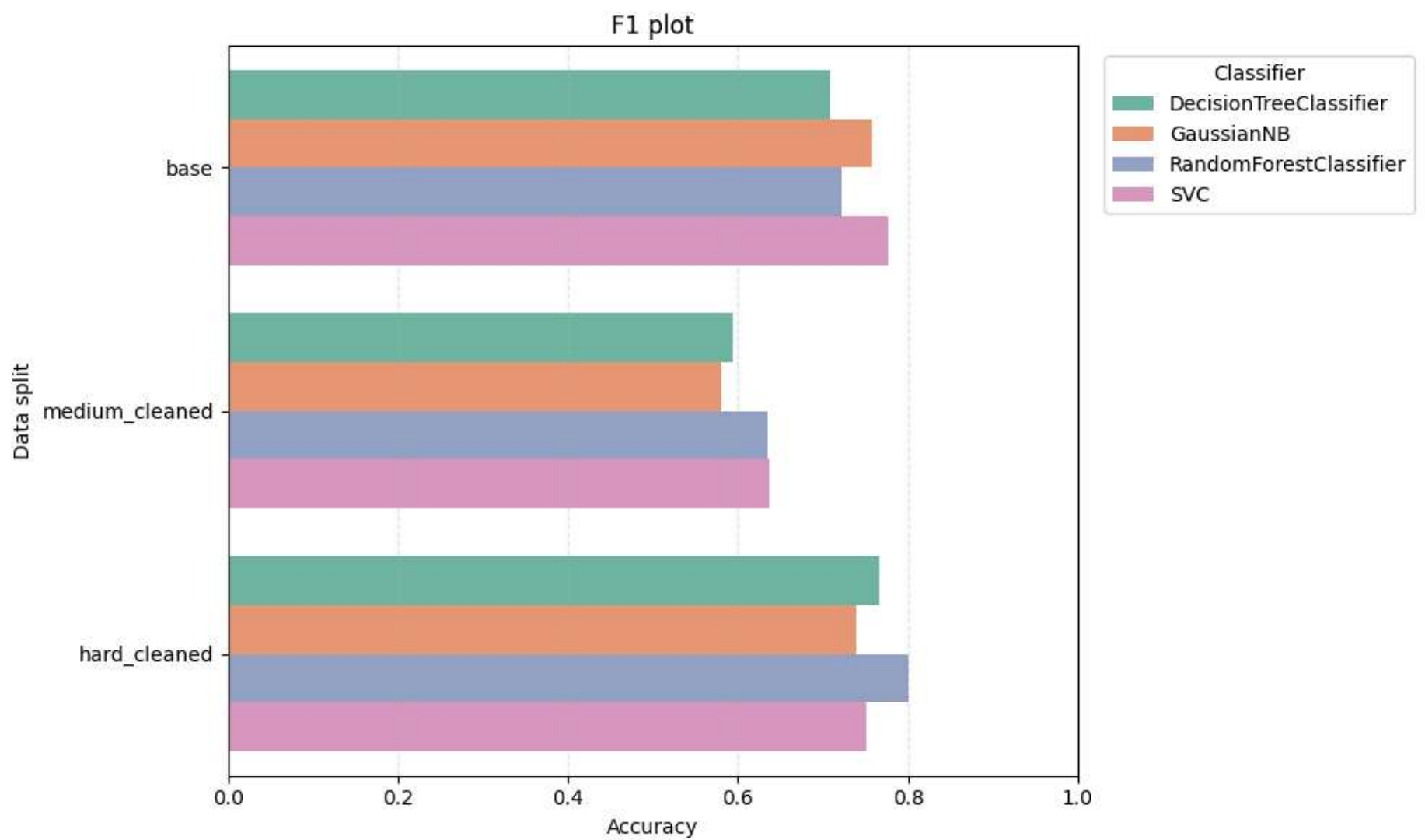
```
ax.set_title(title)
ax.set_xlabel("Accuracy")
ax.set_ylabel("Data split")
ax.set_xlim(0, 1)
ax.legend(title="Classifier", bbox_to_anchor=(1.02, 1), loc="upper left")
ax.grid(axis="x", linestyle="--", alpha=0.3)

plt.tight_layout()
plt.show()

metric_byDataframe_byClassifier(analysis_df, 'Accuracy', 'Accuracy plot', 'Set2')
```



In [59]: `metric_byDataframe_byClassifier(analysis_df, 'F1_Score', 'F1 plot', 'Set2')`



Osiągnięte wyniki prezentują ciekawe własności. Zarówno `Accuracy` jak i `F1 score` najwyższe wartości osiągają dla rygorystycznie wyczyszczonego datasetu. Tak dobre wyniki są spowodowane tym, że modele uczone są na zupełnie kompletnym zbiorze. Nie znaczy to jednak o sile modelu tylko o jego przeuczeniu na małym zbiorze, więc jego użyteczność w rzeczywistym środowisku nie była by zbyt, jako że braki w danych dotyczących badań są raczej standardem.

W bezpośrednim porównaniu klasyfikatorów uwidoczniła się silna zależność między architekturą modelu a strukturą użytego zbioru danych. Na pełnym, najbardziej zaszumionym zbiorze bazowym, zwycięzcą okazał się algorytm SVC (F1-score: 0.7774). Dzięki wykorzystaniu jądra liniowego i silnej regularyzacji, model ten zdołał wytyczyć twarde granice decyzyjne, wykazując się najwyższą odpornością na szum informacyjny wprowadzony podczas imputacji braków.

Zaskakująco wysokie, drugie miejsce na zbiorze bazowym zajął **Naiwny klasyfikator Bayesa** (GaussianNB, F1-score: 0.7573), którym zdeklasował bardziej zaawansowane modele oparte na drzewach. Wynika to z faktu, że sztuczna imputacja braków średnią lub medianą uwypukliła centralne wartości rozkładu, co idealnie wpisało się w założenie tego algorytmu o rozkładzie normalnym danych.

Z kolei zaawansowany **Random Forest**, mimo ogromnego potencjału, poradził sobie na zbiorze bazowym zauważalnie słabiej (F1-score: 0.7229). Wprowadzenie ogromnej liczby identycznych, sztucznych wartości (median) utrudniło algorytmom drzewiastym odnajdywanie optymalnych punktów podziału dla gałęzi. Moc Random Forest widać dopiero w ewaluacji na zbiorze hard_cleaned, gdzie w środowisku pozbawionym sztucznie łątanych luk informacyjnych, algorytm ten osiągnął najwyższy wynik w całym zestawieniu (F1-score: 0.8007), wyprzedzając SVC. Potwierdza to, że modele ensemble brylują na danych czystych, lecz w środowisku wysokiej niepewności i silnej imputacji ustępują stabilności **SVC**

Analizując uzyskane modele pod kątem rzeczywistego zastosowania faworytem wydaje się być klasyfikator **SVC** dla zbioru bazowego:

```
SVC,median,constant,0.7976,0.7598,0.7976,0.7774,{ 'classifier__C': 10, 'classifier__gamma': 'scale',
'classifier__kernel': 'linear', 'preprocessor__cat__imputer__strategy': 'constant',
'preprocessor__num__imputer__strategy': 'median', 'preprocessor__num__scales': StandardScaler()},base
```

Osiąga **accuracy** i **f1** porównywalne z overfitowanymi modelami trenowanymi na danych z hard_cleaned datasetu i jest znacznie bardziej użyteczny, ponieważ jest trafnie w stanie obsługiwać pacjentów z niepełną historia. Jego wysoka skuteczność wynika z odporności wektorów nośnych na uśredniony szum wprowadzany przez imputację. Co więcej, wybór jądra liniowego oraz wysokiej kary za błędy gwarantuje wysoką interpretowalność modelu i jego wyczulenie na rzadkie, krytyczne stany medyczne pacjentów.

3. Porównanie SVC dla zbioru base i hard cleaned

```
=== SVC - base ===
Hiperparametry: {'classifier__C': 10, 'classifier__gamma': 'scale', 'classifier__kernel': 'linear',
'preprocessor__cat__imputer__strategy': 'constant', 'preprocessor__num__imputer__strategy': 'median',
'preprocessor__num__scales': StandardScaler()}

--- WYNIKI DLA CAŁEGO ZBIORU TESTOWEGO ---
      precision    recall  f1-score   support

      C         0.80      0.89      0.84         44
      CL         0.00      0.00      0.00          4
      D         0.80      0.78      0.79         36

 accuracy         0.80         84
macro avg         0.53         0.55         0.54         84
weighted avg         0.76         0.80         0.78         84

--- WYNIKI TYLKO DLA MĘŻCZYZN ---
      precision    recall  f1-score   support

      C         1.00      1.00      1.00          2
      D         1.00      1.00      1.00          5

 accuracy         1.00          7
macro avg         1.00      1.00      1.00          7
weighted avg         1.00      1.00      1.00          7

=== SVC - hard_cleaned ===
Hiperparametry: {'classifier__C': 10, 'classifier__gamma': 'scale', 'classifier__kernel': 'rbf',
'preprocessor__cat__imputer__strategy': 'most_frequent', 'preprocessor__num__imputer__strategy': 'mean',
'preprocessor__num__scales': MinMaxScaler()}

--- WYNIKI DLA CAŁEGO ZBIORU TESTOWEGO ---
      precision    recall  f1-score   support

      C         0.74      0.93      0.82         30
      CL         0.00      0.00      0.00          0
      D         0.88      0.54      0.67         26

 accuracy         0.75         56
macro avg         0.54         0.49         0.50         56
weighted avg         0.80         0.75         0.75         56

--- WYNIKI TYLKO DLA MĘŻCZYZN ---
      precision    recall  f1-score   support

      C         0.50      0.33      0.40          3
```

CL	0.00	0.00	0.00	0
D	0.67	0.67	0.67	6
accuracy			0.56	9
macro avg	0.39	0.33	0.36	9
weighted avg	0.61	0.56	0.58	9

Pierwsze co rzuca się w oczy przy porównaniu modeli jest mała liczba osób o `Status = CL` . W początkowym zbiorze około 20 osób, a do zbiorów testowych trafiło odpowiedni 4 i 0 osób o takim statusie. W przypadku zbioru bazowego, przy tak znikomej reprezentacji, model nie był w stanie poprawnie sklasyfikować tych przypadków i przypisywał ich do grup `C` lub `D` (Precision i Recall wynoszące 0.00). Co jednak kluczowe, w zbiorze `hard_cleaned` klasa CL całkowicie zniknęła z próby testowej.

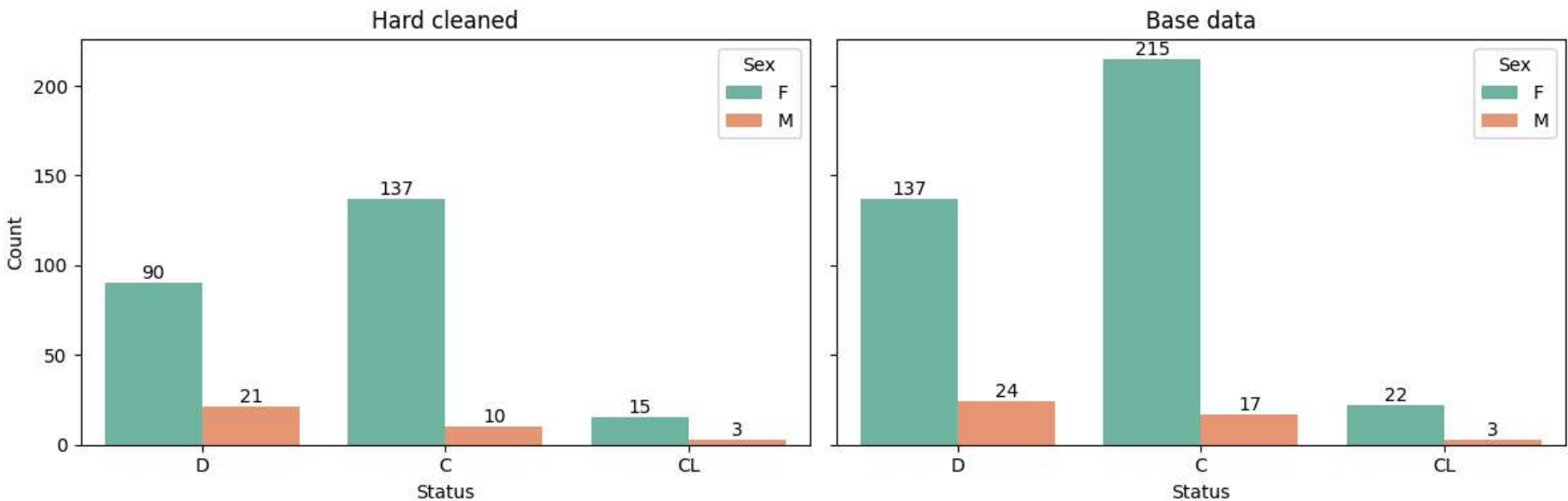
```
In [60]: fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

sns.countplot(data=df_cleaned_more, x='Status', ax=axes[0], palette='Set2', hue = 'Sex')
axes[0].set_title('Hard cleaned')
axes[0].set_xlabel('Status')
axes[0].set_ylabel('Count')

sns.countplot(data=df, x='Status', ax=axes[1], palette='Set2', hue = 'Sex')
axes[1].set_title('Base data')
axes[1].set_xlabel('Status')
axes[1].set_ylabel('')

for ax in axes:
    for container in ax.containers:
        ax.bar_label(container)

plt.tight_layout()
plt.show()
```



Dowodzi to, że pacjenci wymagający przeszczepu mieli niekompletne wyniki badań, że usuwanie braków danych zmniejszyło tą próbkę osób o 40%.

Co ciekawe model trenowany na bazowych danych poradził sobie ponad przeciętnie dobrze, idealnie klasyfikując wszystkich mężczyzn którzy znaleźli się w zbiorze testowym. Model trenowany na obciętym zbiorze danych poradził sobie trochę gorzej, jednak należy mieć na uwadze, że liczba rekordów jest za mała, aby uznać to za rzetelny test modelu

Ciekawym okazał się również fakt doboru optymalnych hiperparametrów i metod transformacji przez optymalizator `GridSearchCV`. W przypadku zbioru bazowego (base), algorytm preferował jądro liniowe (kernel: 'linear') połączone z bardzo silną regularyzacją (C: 10) oraz standaryzacją danych `StandardScaler`. Oznacza to, że po uzupełnieniu braków medianą, zachowanie naturalnych proporcji wariancji (bez drastycznego spłaszczania przez wartości odstające) pozwoliło modelowi wytyczyć twarde, proste granice decyzyjne, silnie karząc się za każdy błąd klasyfikacyjny.

Z kolei dla drastycznie uciętego zbioru `hard_cleaned`, model został 'zmuszony' do użycia nieliniowego jądra (kernel: 'rbf') – zachowując przy tym równie wysoką karę (C: 10) – oraz normalizacji `MinMaxScaler`. Taki dobór parametrów sugeruje, że rygorystyczne usunięcie pacjentów z brakami danych pozostawiło w zbiorze specyficzną, trudniejszą do geometrycznego rozdzielenia grupę przypadków. Jądro RBF, opierające się na dystansach, wymagało ścisłego zamknięcia wszystkich cech w równych, znormalizowanych przedziałach, aby móc poprawnie zmapować pacjentów do przestrzeni o wyższym wymiarze. W

Wnioski Końcowe

Zbiór pacjentów okazał się dość ciężki do analizy, jednak nie ze względu na braki lub dystrybucje lecz rozmiar. 400 rekordów to absolutnie mikroskopijny zbiór w porównaniu do standardowych zbiorów wykorzystywanych do ćwiczenia mechanizmów uczenia maszynowego. Dodatkowo ciężko jest wyciągać sensowne wnioski przez ilość rekordów np. sytuacje w których do zbioru testowego nie trafia żadna osoba os statusie CL.